

Combining Motion Planner and Deep Reinforcement Learning for UAV Navigation in Unknown Environment

Yuntao Xue^{ID} and Weisheng Chen^{ID}

Abstract—Navigation of unmanned aerial vehicles (UAVs) in unknown environments is a challenging problem, and it is worth considering how to reach the target through static obstacles in a **safe and energy-efficient** manner. The traditional motion planning algorithm is easy to get into trouble when the obstacles are dense. **The navigation algorithm based on reinforcement learning has better generalization and robustness, but the trajectory generated by the end-to-end method is not smooth and dynamic enough.** In this work, a classical motion planning algorithm and deep reinforcement learning (DRL) algorithm are combined named RLPlanNav, which aims to solve the problem of safe and dynamic navigation of UAVs in unknown environments. **The upper-layer DRL algorithm part of the framework receives the sensor raw information to generate the next local target, and the lower-layer classical planner generates a smooth and safe trajectory to reach the target.** The DRL algorithm incorporates an LSTM network to add memory capabilities, thereby ensuring the effectiveness of local target selections. The proposed navigation framework is tested in a simulated environment where static obstacles are randomly generated, and has higher navigation success rates and more kinematic-compliant navigation trajectories compared to traditional motion planning methods and end-to-end methods.

Index Terms—Deep reinforcement learning, motion planning, partially observable Markov decision process, UAV navigation.

I. INTRODUCTION

IN THE past few decades, autonomous UAVs have occupied an important position in the fields of emergency rescue and logistics distribution due to their flexible mobility, light size and multi-functionality [1]. Smart UAVs can replace humans to perform many dangerous tasks, and the ability to navigate autonomously is a prerequisite for UAVs to complete various tasks. Traditional autonomous navigation consists of multiple modules including positioning and mapping, global path planning and local path planning, which receive sensor information to estimate the state of the UAV and make planning decisions. Global path planning and local path planning need to be specially

configured for the environment, and the process of building an environment map also consumes a lot of computing resources. Therefore, it is of great significance to study a map-free motion planner with good generalization ability and adaptability to complex environments.

With the development of deep reinforcement learning, UAVs for navigation based on DRL can directly map sensor raw information to control signals, and can perform safe navigation tasks in unknown environments without map prior information. The decision-making strategy of navigation is obtained through interactive training in various environments, which can be effectively generalized to various environments. Mnih et al. [2] used the DQN algorithm to control video games in discrete motion spaces, and then Lillcrap et al. [3] designed a deep deterministic gradient algorithm (DDPG) applied to continuous action spaces based on the actor-critic framework. Heess et al. [4] designed a recurrent deterministic policy gradient algorithm considering some considerable environments, and Pham et al. [5] designed a RL-based algorithm for UAV navigation in unknown environments. However, the trajectory generated by the UAV control strategy trained by the end-to-end (E2E) method cannot be as smooth and dynamic as the traditional planning algorithm. Its control curve is relatively rigid, which will increase energy consumption.

For smooth, energy-efficient, kinetic UAV navigation in unknown environments full of complex static obstacles, and instant escape from local traps to reach the target point. We propose RLPlanNav, a hierarchical navigation framework that combines DRL and classical motion planners, using reinforcement learning in the upper layer to make high-level decisions in an unknown environment, and a classical motion planning module in the lower layer responsible for generating smooth trajectories. Specifically, a DRL training policy is used, which progressively generates a series of local landmarks using the current RGB image observations. These local landmarks are passed to the motion planner to generate precise collision-free control sequences for execution on the UAV. In the DRL module recurrent deterministic policy gradient algorithm, which adds a memory module for some considerable environments, so that the UAV can escape the local trap and complete the navigation. The motion planner in the framework adopts the gradient-based EGO-planner, which has the characteristics of lightweight calculation and fast speed, and can efficiently complete the re-planning. The proposed navigation framework is called RLPlanNav and is summarized in Fig. 1.

Manuscript received 10 July 2023; accepted 14 November 2023. Date of publication 20 November 2023; date of current version 1 December 2023. This letter was recommended for publication by Associate Editor B. Duncan and Editor P. Pounds upon evaluation of the reviewers' comments. This work was supported by the National Natural Science Foundation of China under Grants 62073254 and 92271101. (Corresponding author: Weisheng Chen.)

The authors are with the School of Aerospace Science and Technology, Xidian University, Xi'an 710071, China (e-mail: xueyuntao@stu.xidian.edu.cn; wshchen@126.com).

Digital Object Identifier 10.1109/LRA.2023.3334978

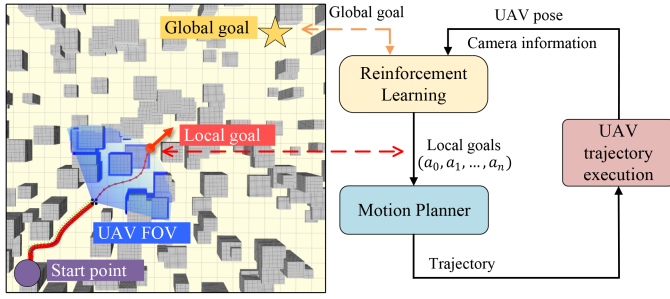


Fig. 1. Proposed RLPlanNav framework consists of an upper DRL-based decision module and a motion planner-based planning module. The decision module predicts a local target based on current sensor information. This local goal is received by the motion planner-based planning module and generates a smooth, dynamically feasible trajectory to the local goal. This process repeats until the UAV reaches the final target.

The contributions of this letter are as follows:

- 1) By combining a DRL-based approach and a motion planner-based approach, a navigation framework is proposed that leverages the strengths of each to help UAVs navigate in unknown environments.
- 2) The UAV navigation problem in the unknown environment is established as an improved POMDP model, and the sub-goal generation policy is obtained by training the deterministic policy gradient algorithm to solve the problem.
- 3) By performing performance tests on the RLPlanNav algorithm in an environment where static obstacles are randomly generated, it is proved that the proposed navigation framework outperforms the previous methods in trajectory quality.

The rest of this letter is structured as follows. Related work is discussed in Section II. Section III presents the navigation model and problem formulation. The hierarchical UAV navigation framework and recurrent deep policy gradient algorithm are introduced in Section IV. Section V evaluates the algorithm performance in experiments. Section VI concludes the letter.

II. RELATED WORK

A. Classical Motion Planning

The basic modules required for autonomous UAV navigation include positioning and mapping, path search, and trajectory generation. The goal of path search has been to find a safe path composed of discrete points from the initial point to the target point, which can be divided into search-based methods such as A* and Dijkstra's algorithm [6], and sampling-based methods such as Probabilistic Road-Map (PRM) [7] and Rapidly-Exploring Random Tree (RRT) [8]. The trajectory obtained by the path search algorithm does not contain time information. In order to ensure the feasibility of UAV motion dynamics, the initial trajectory needs to be parameterized in time to facilitate control. Gao et al. [9] used a 3D laser rangefinder to gradually build a point cloud map and generates trajectories directly. Oleynikova et al. [10] combined the concept of local exploration and local planner for UAV navigation in unknown forests. In our work, a generative policy that generates local objects be trained by DRL with a higher success rate than classical heuristic methods for finding local points.

B. RL for Navigation

With advancements in deep learning and reinforcement learning, significant focus has turned to E2E learning for training navigation strategies in mobile robots. Zhu et al. [11] employed an actor-critic model to learn policies from raw observations, needing minimal fine-tuning for real robot adaptation. For UAV navigation, [12] decomposed the task into two LSTM-based DRL networks, effectively handling obstacle avoidance and target acquisition. The inclusion of LSTM enhances dynamic obstacle information acquisition, improving UAV safety. However, their two-dimensional simulation environment may not fully capture three-dimensional UAV flight characteristics. Another algorithm [5] focused on training point navigation tasks in discrete environments on a physical UAV using PID and Q-learning, but the discrete state space assumption limits practical applicability, resulting in straight-line trajectories. UAV motion planning in continuous space has also been studied. Kim et al. [13] applied a proximal policy optimization algorithm for UAV motion planning in the Gazebo simulator, involving static obstacles, yielding an 81% goal achievement rate. Likewise, [14] introduced the relevant experience learning-DDPG (REL-DDPG) algorithm, amalgamating double screening sampling and prioritized experience replay to enhance UAV autonomous motion planning. However, trajectories from E2E RL methods may not ensure dynamic feasibility and path smoothness.

C. RL With Motion Planner

Recent research aims to merge learning and traditional navigation methods for enhanced robot autonomy. In [15], a residual policy augments classical controllers, and can switch to classical control when policy performance is uncertain in complex indoor navigation tasks. Faust et al. [16] utilized RL to learn short-range local navigation strategies, utilizing probabilistic roadmaps (PRMs) for long-range navigation. These methods share a reliance on RL for control policy, demanding substantial computational resources due to high-dimensional states. To maximize decision-making potential, [17] introduced a learning-based module for waypoint generation and a model-based planner for smooth, feasible trajectories, known as LB-WayPtNav, tested on ground mobile robots. In [18], an RL policy was used to generate sub-goals from high-dimensional observations, with a flexible motion planner for adaptable robot navigation in various environments. Notably, these algorithms target ground robots. Inspired by this approach, we propose a high-dimensional UAV navigation framework, integrating reinforcement learning for local targets and classical motion planners for trajectory planning.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. UAV Model

We ignore the momentum of the UAV during flight, assuming that the steering action takes effect immediately. The dynamics equation for a UAV in a 3D environment is expressed as:

$$\begin{cases} \vartheta_{t+1} = \vartheta_t + \rho_t \\ \varphi_{t+1} = \varphi_t + \tau_t \\ x_{t+1} = x_t + v \times \cos(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ y_{t+1} = y_t + v \times \sin(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ z_{t+1} = z_t + v \times \sin(\varphi_{t+1}) \end{cases} \quad (1)$$

where ϑ_t and φ_t are the directional angles of the UAV in the horizontal and vertical planes, respectively, ρ_t and τ_t are the steering signals of the two angles, respectively, and v is UAV velocity.

B. Problem Description

This work aims to safely and efficiently navigate UAVs in unknown complex environments. Safety ensures collision-free navigation, while efficiency focuses on finding optimal paths respecting the UAV's dynamics. The primary challenge arises from dense static obstacles and the absence of prior maps, prevalent in environments like forests or densely built urban areas. The UAV's limited observability stems from cameras or range sensors with constrained field of view (FOV) in both horizontal and vertical directions, a realistic assumption. We assume accurate perception with an RGB-D camera or range sensor, and a static environment.

The objective of this letter is to enable a UAV to safely and smoothly navigate from an arbitrary starting point $p_0 = (x_0, y_0, z_0)$ to a target position $p_g = (x_g, y_g, z_g)$ in an unknown complex environment without collisions. At each time step t , the UAV receives environmental and pose information, collectively called observation o_t . The goal is to design a strategy that utilizes the UAV's observation o_t to generate local goals, which are used by the motion planner to generate dynamically feasible trajectories. This iterative process guides the UAV efficiently towards the target. When the UAV perceives an obstacle ahead at location P_2 , the DRL-based policy generates a local goal $p_l = (x_l, y_l, z_l, \phi_l, \theta_l, \psi_l)$ (represented by a red dot for position and red arrow for direction) to navigate away from the predicament.

The navigation performance of the UAV is evaluated based on the success rate, average time to reach the target, and average acceleration and jerk along the trajectory.

IV. RLPLANNAV: RL WITH MOTION PLANNER

A. UAV Navigation as a POMDP

The UAV navigation task is formulated as a time-discrete partially observable Markov decision process (POMDPs), which is a 7-tuple $(S, A, T, R, \Omega, O, \gamma)$. where S represents the state set, A represents the action set, T represents the state transition probability, $R : S \times A \rightarrow \mathbb{R}$ represents the reward function, Ω represents the observation set, O represents the conditional observation transition probability, and $\gamma \in [0, 1)$ represents the discount factor. The environmental state at time step t is $s \in S$, the agent takes an action $a \in A$, and the environmental state will transition according to the state transition probability $T(s_{t+1} | s_t, a_t)$ transfers to the next state s_{t+1} , the agent receives a new observation $O(o_t | s_{t+1}, a_t)$, and get reward r_t at the same time. The agent's goal is to choose actions at each time step that maximize the expectation of future discounted rewards: $E[\sum_{t=0}^{\infty} \gamma^t r_t]$. The agent cannot directly observe the state s_t , and can only observe the transition probability $O(o_t | s_{t+1}, a_t)$ from the observation set Ω according to the condition estimate potential states.

The action set A can be directly set as the control signal of the continuously controlled UAV, such as velocity and

angular velocity. However, based on the above mentioned, directly training the UAV control signal makes the generated trajectory not smooth enough, and the convergence speed is slow. So the consideration in RLPlanNav is to assume that a successful policy can be described as a sequence of local goals: $\tau_{sus} = \{p_0, \dots, p_{\tau-1}\}$, each of which corresponds to the configuration space of the motion planner, representing the pose information in the 3D environment. The navigation algorithm should query the motion planner at each policy step to generate smooth collision-free trajectories. The motion planner takes as input the local targets generated by the DRL strategy and outputs an action sequence that satisfies the dynamic feasibility. The RLPlanNav framework consists of two components, a local target generation strategy and a motion planner.

The local target generation strategy accepts the sensor raw observation information o_t and outputs the action a_t , which is the local target of the motion planner. The combination of the trained local object generator policy and MP is a policy to solve POMDP. The proposed RLPlanNav solves this POMDP problem using the recurrent policy gradient algorithm.

B. Recurrent Deep Deterministic Policy Gradient

Deep deterministic policy gradient algorithm is used to solve continuous control problems in fully appreciable environments. The DDPG algorithm adopts the actor-critic framework. The actor network is responsible for interacting with the environment to generate a deterministic policy with θ parameters μ^θ , and the critic network is based on the action-value function related to the current policy, that is, the Q function Q^μ evaluates the actor's performance and guides the policy to improve. Its policy can be updated via backpropagation:

$$\nabla \eta(\mu) = \mathbb{E}_{s \sim \rho_\mu} \left[\nabla_\theta \mu(s) \nabla_a Q_\mu(s, a) \Big|_{a=\mu(s)} \right] \quad (2)$$

where ρ^μ is the discounted state access distribution within an episode for the current policy μ .

In a partially observable environment, the optimal policy needs to rely on the information of the previous historical trajectory h_t . By replacing the feedforward network with a recurrent neural network, the network can learn the relevant information before. Therefore, it is necessary to replace $\mu(s)$ and $Q(s, a)$ with $\mu(h)$ and $Q(h, a)$ accordingly, and the policy update becomes:

$$\nabla \eta(\mu) = \mathbb{E}_\tau \left[\sum_t \gamma^{t-1} \nabla_\theta \mu(h_t) \nabla_a Q_\mu(h_t, a) \Big|_{a=\mu(h_t)} \right] \quad (3)$$

where the expectation corresponds to the entire trajectory $\tau = (s_1, o_1, a_1, s_2, o_2, a_2, \dots)$, and $h_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ is the observation-action trajectory history at time step t . In RLPlanNav, the action a_t becomes the generated local target pose p_t . In the actual learning process, we introduce experience replay to improve data efficiency, and use the target network to enhance the stability of the algorithm. Refer to Section IV for the actual operation.

1) *Observation Space*: The local target generation strategy (LoGP) receives the sensor raw information and outputs MP sub-targets (as shown in Fig. 2). In order to successfully navigate to

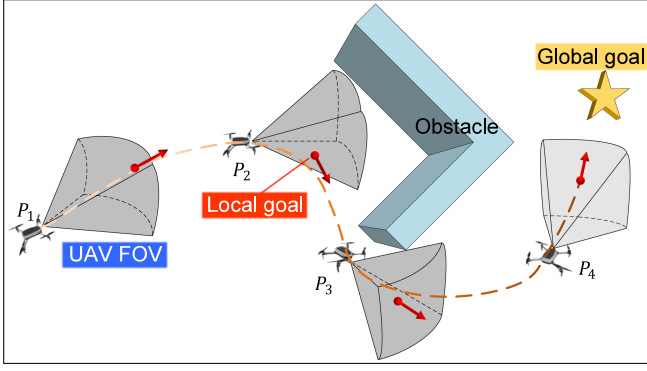


Fig. 2. Schematic illustration of a UAV with a limited field of view relying on local target guidance in an unknown environment.

the target, the UAV needs to receive three aspects of observation information, namely the UAV's own pose information (including velocity and angular velocity), the relative relationship with the target, and the relationship with the environment. The relative position relationship between the UAV and the target is given at the initial moment, and the relationship between the UAV and the environment is observed by the onboard RGB depth camera. The output is the desired next local target.

In the field of UAV navigation, some research tends to simplify the images to achieve compression of the information thus reducing the arithmetic power while preserving the features [19]. Since models based on **Variational Autoencoders (VAEs)** [20] can typically extract rich and compact visual perception information from images, we attempted to combine VAE as an auxiliary role in unsupervised learning with policy-based DRL algorithms in this study. The input depth map is an 84×84 grayscale image with channel index 1, represented as $\mathcal{I}$. When the VAE converges, the latent vector generated in the VAE encoding part is a 50-dimensional code and retains the original depth map features. The observation space of the UAV constructed using this method is represented as $o = [\vartheta, \varphi, \mathcal{I}, \xi]$.

2) *Action Space*: The generation strategy of the local target parameterized by continuous action is called LoGP. The high-level observation regards the space of the local target as a continuous action space, and the policy network predicts a vector $a = [x, y, z, \phi, \theta, \psi]$, that is, a position information in the body coordinate system and orientation information to represent the final end effector pose after interaction. The variables ϕ, θ, ψ correspond to the direction information of the predicted local target point. This direction information is utilized to refine the motion trajectory. The local target generation strategy is trained by RDPG, and the resulting local targets are executed by the motion planner ego-planner introduced in the next section.

3) *Reward Design*: In order for the agent to gradually learn to make reasonable decisions about the observed information, an appropriate reward function must be chosen as the feedback of the action. By incorporating prior knowledge in the navigation domain, a non-sparse reward is designed, which includes distance reward and obstacle penalty. The distance reward is expressed as:

$$r_{dist} = \omega_g (\|p^{t+1} - g\| - \|p^t - g\|) \quad (4)$$

where p^t represents the three-dimensional space coordinates of the UAV at t , and g represents the three-dimensional space coordinates of the target point. This reward represents the shortened distance from the final target after the UAV executes the trajectory to the local target through the motion planner. Besides, the chosen local target should be in free space to keep the UAV safe, so an obstacle penalty is imposed to avoid collision, which is designed as:

$$r_{obs} = -e^{-\sigma d_{min}} \quad (5)$$

where d_{min} represents the Euclidean distance between the local target of the decision and the nearest obstacle. The total reward is expressed as:

$$r = r_{dist} + r_{obs} \quad (6)$$

C. Motion Planner

The motion planner of the proposed RLPlanNav framework adopts the ESDF-free gradient-based EGO-planner proposed by Zhou [21], which stores the obstacle information only when the trajectory collides with a new obstacle, and constructs ESDF to completely serve the trajectory optimization, so as to ensure Good trajectory optimization with less computation.

EGO-planner performs trajectory optimization in two steps: first the trajectory is optimized using smoothness, collision and dynamic feasibility, and if the resulting trajectory violates the dynamic limits, the trajectory time is redistributed to perform the refinement process.

First, the trajectory is parameterized by a uniform B-spline curve, and the trajectory contains N control points $\{Q_1, Q_2, \dots, Q_N\}$ and a knot vector $\{t_1, t_2, \dots, t_M\}$, where $Q_i \in \mathbb{R}^3, t_m \in \mathbb{R}$. Follow [Minimum snap trajectory generation and control for quadrotors] to reduce control points to a differentially flat output space. The optimization problem can be modeled as:

$$\min_Q J = \lambda_s J_s + \lambda_c J_c + \lambda_d J_d \quad (7)$$

where J_s is the smoothing penalty, J_c is the collision penalty, J_d is the dynamic feasibility term, $\lambda_s, \lambda_c, \lambda_d$ are each penalty term previous weight.

In the unknown environment, new static obstacles are constantly added when exploring and moving forward, so the planner needs to be restarted quickly. In order to quickly solve the Hessian information of the objective function, the quasi-Newton method is used to approximate the Hessian from the gradient. Then an anisotropic curve fitting algorithm is introduced, so that the higher-order derivatives of the trajectory can be adjusted while maintaining the original shape, and finally a dynamically feasible trajectory with reasonable time allocation is generated.

V. EXPERIMENTAL RESULTS

A. Simulation Setup and Compared Methods

The simulated environment, as shown in Fig. 3, is generated in the ROS environment. It encompasses four distinct environments, namely randomly generated square pillars, columns, circles, and irregularly shaped foggy 3D obstacles, effectively

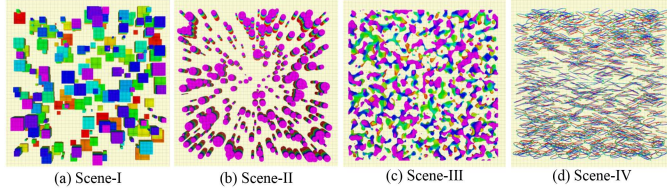


Fig. 3. Navigation algorithm will be tested in the following four environments, and trained by randomly selecting scenes and starting point pairs. The environment includes square column obstacles, cylindrical obstacles, foggy obstacles, and circular obstacles.

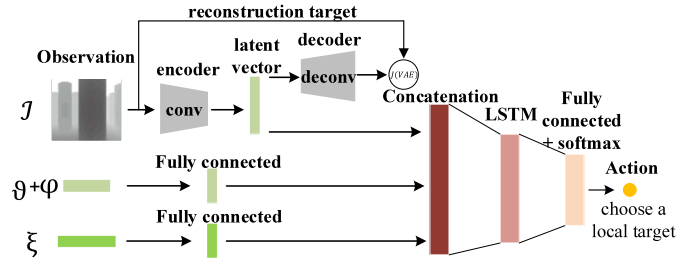


Fig. 4. Architecture of actor networks to select actions.

simulating urban buildings, forest trees, and other relevant obstacles. The initial pose of the UAV, the locations of environmental obstacles and targets are randomized during training. The trained policy will be tested in scenarios with different obstacle density and location.

The hyperparameters tuned through multiple rounds of experiments are used to deploy the model. The parameters of the reward are set as: $\sigma = 2.0$, $\alpha = 8.0$, $\beta = 25.0$, $r_{free} = 0.1$, $r_{step} = -0.6$. The longest historical estimate length is limited to 100. The experiment assumes that the learning rate of Adam Optimizer is 0.01, and the update rate of the target network is $\tau = 0.01$. The size of the replay buffer is 10^5 and is sampled every 100 time steps. The discount factor is $\gamma = 0.99$. The structure of the actor network is shown in Fig. 4.

The proposed method will be compared to the following baseline way to evaluate the efficiency of the model in this research. The same states, actions, and rewards are used in all baselines.

a) *End-to-end approach*: Three E2E methods are used for comparison, namely RDPG [4], PPO [13] and REL-DDPG [14]. This purely learning-based approach requires no explicit system knowledge. The action space for the RL method is derived from control signals, utilizing a vector $a = [\rho, \varphi]$ where ρ represents the throttle signal and φ denotes the steering signal within the range of $-\pi$ to π . These two signals constitute the UAV's action space.

b) *Non-learning based approach*: This approach progressively builds an occupancy grid of the environment from depth images, and then generates velocity controls through a spine-based planner. A segment of the path planned by the A* algorithm is generated from the starting point, and the local trajectory is continuously replanned using the ego-planner as the UAV progresses. If the local planner becomes trapped, a new path is generated using the globally planned A* algorithm in an attempt to escape the dilemma.

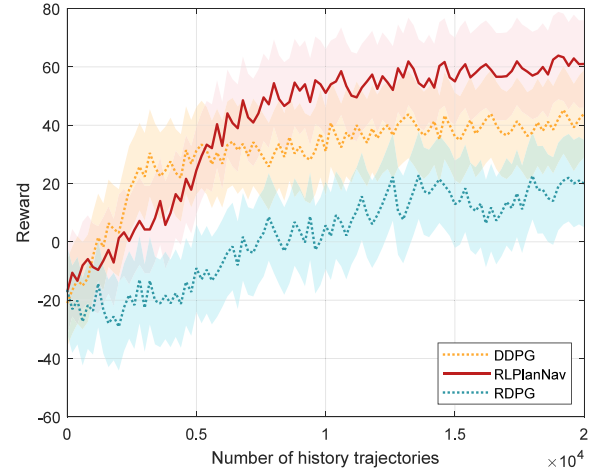


Fig. 5. Reward during all the training episodes in Scene-I.

B. Evaluation Metric

A set of quantitative evaluation measures that indicate the safety, efficacy, and robustness of the navigation algorithm were designed to evaluate the quality of UAV navigation. The following are the details:

a) *Average reward*: The learned policy completes 250 navigation tasks in four environments, with the reward being the average of all times and agents. Smaller rewards imply a lower level of performance.

b) *Success rate, Trapped rate, Collision rate*: Refers to the percentage of agents that successfully reached the goal point, failed to reach the goal point, and collided within the allotted time, respectively.

c) *Path efficiency*: The ratio of the sum of the straight-line distance between the starting point and the target point of all agents to the sum of their the actual trajectory lengths.

d) *Average acceleration and average jerk*: These metrics assess the performance in terms of smoothness and energy consumption during the navigation task.

C. Performance of RLPlanNav

a) *Average return*: The average training returns for RLPlanNav, RDPG, and DDPG are shown in Fig. 5. Specifically, the cumulative rewards accumulated by the agents over 250 training calendars were averaged. The results show that our proposed RLPlanNav algorithm has better sampling efficiency and rewards compared to the DDPG and RDPG algorithms. Both in terms of convergence speed and average total reward after convergence, the RLPlanNav algorithm outperforms the RDPG algorithm. The RLPlanNav algorithm reaches convergence in 9000 rounds, which is 4000 rounds faster than the RDPG. The speed improvement is due to the fact that the RLPlanNav algorithm can focus on finding the local points where the reward can be maximized without learning basic flight rules and skills. In addition, the stability of the algorithm also plays a key role in the performance of the agent. As can be seen from Fig. 5, the RLPlanNav algorithm and the RDPG algorithm are more stable than the DDPG algorithm. Our experimental environment is full of unknown and complex obstacles, and both recursive

TABLE I
FLIGHT TIME AND PATH EFFICIENCY OF THE RLPlanNav ALGORITHM AFTER
100 FLIGHT TESTS IN FOUR SCENES

Flight scene	Flight time (s)	Path efficiency (%)
SCENE-I	49.84	78.15
SCENE-II	40.16	86.71
SCENE-III	41.79	85.29
SCENE-IV	39.85	87.53

algorithms can utilize memory to memorize information about these partially observable environments to obtain stable rewards.

b) Performance under different map structures: Different map structures can have a significant impact on flight performance. We conducted 100 successful flight tests in four scenes with an obstacle density of 50% and a scene side length of $50 \text{ m} \times 50 \text{ m}$ to examine their effects. The results, displayed in Table I, reveal that the RLPlanNav algorithm consistently maintains flight times around 40 seconds with a general flight speed of approximately 1.5 meters per second in scenarios featuring cylindrical, fog-like, and ring-shaped obstacles.

Notably, the most notable deviation in flight time occurs in the presence of square column obstacles. Flight times in this scenario are about 10 seconds longer compared to the other three environments. This disparity is attributed to square column obstacles, which, at higher densities, can hinder UAVs' visual perception due to their right-angled edges, leading to challenging situations. Hence, in the cuboid obstacle scenario, the RLPlanNav algorithm experiences longer flight times. Similarly, the efficiency of the flight path in the cuboid obstacle scenario is approximately 7% lower than in the other three scenarios, indicating that the UAV needs to navigate a more intricate route to reach its destination in this environment.

D. Compared With Reinforcement Learning Methods

From the results in Table II, the success rate of the agent using the RLPlanNav algorithm to navigate to the target point surpasses the E2E RDGP method by 15%. Moreover, the time required to reach the goal is 24% shorter, and the average acceleration is 29% lower, indicating reduced energy consumption. The trajectory graphs in Fig. 6 highlight the methods' operation based on onboard camera sensor data. RLPlanNav employs a model-based local planner for back-end processing, focusing on selecting a suitable local target point for the underlying model-based planner. This enables the UAV to navigate through narrow passages and turn at corners composed of obstacles. In contrast, RDGP methods face greater challenges in this regard.

The speed control curve in Fig. 7 reveals that despite the addition of an acceleration change limit in the E2E RDGP method's penalty term, the speed control curve during testing remains discontinuous and abrupt. While adding more restrictions to the acceleration component in the RL method's reward function may enhance trajectory smoothness, it leads to a corresponding decrease in success rate. Balancing smooth trajectories and successful navigation is more challenging. In contrast, RLPlanNav's reinforcement learning algorithm doesn't concern itself with trajectory smoothing and execution issues, achieving a smooth dynamic trajectory through a model-based

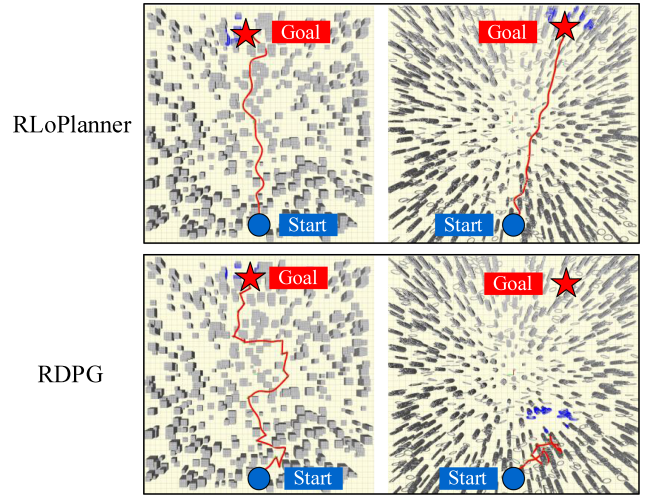


Fig. 6. Schematic diagram of the navigation trajectory of the RLoPlanner agent trained in the simulation environment. The blue circles and red pentagrams represent the starting point and target point of an arbitrary choice, respectively. The curve connecting it represents the trajectory of the UAV.

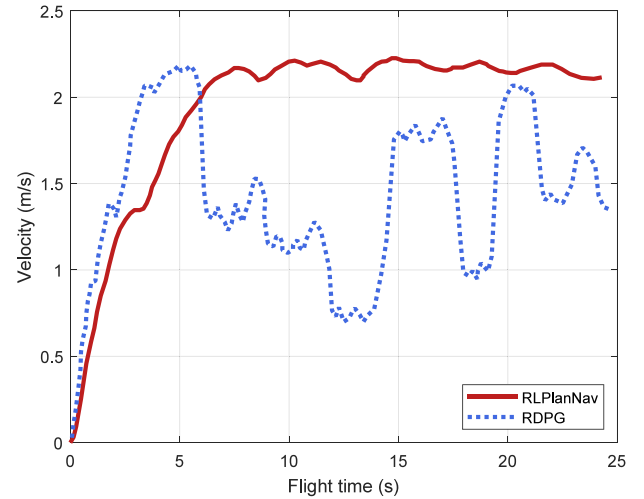


Fig. 7. As the flight time changes, the speed changes of the RLoPlanner algorithm and the RL algorithm during navigation.

control method with lower acceleration changes. These traits are also reflected in Table II. Endurance is a critical bottleneck for UAVs, and the low acceleration rate in RLPlanNav conserves energy, allowing the UAV to cover more distance with the same power.

E. Comparison With Mapping and Planning Methods

Traditional mapping and planning methods have been studied a lot to enable UAVs to autonomously navigate from a starting position to a target position. The mapping and planning approach in this letter uses RGB-D images acquired by onboard cameras to output an occupancy map, which is used by a gradient-based motion planner to output control commands. The navigation results comparison between the proposed RLPlanNav method and the mapping and planning method is shown in Table II. It can be seen that the RLPlanNav method is superior to the mapping and planning method in terms of success rate and trapping

TABLE II
COMPARISON OF THE PROPOSED METHOD WITH E2E METHODS AND CLASSICAL METHODS

Method	Success rate (%)	Collision rate (%)	Trapped rate (%)	Flight time (s)	Acceleration (m/s^2)	Jerk (m/s^3)
RLPlanNav	93.5	5.25	1.25	19.45 ± 3.72	0.14 ± 0.01	3.65 ± 0.13
RDPG(E2E)	86.25	10.25	3.5	28.94 ± 4.18	0.36 ± 0.02	6.1 ± 0.42
EGO-Planner	84.5	9.75	5.75	20.79 ± 2.85	0.13 ± 0.01	3.52 ± 0.16

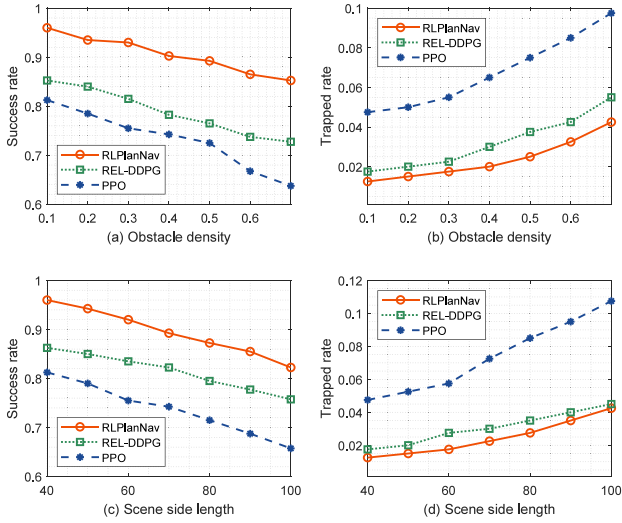


Fig. 8. Navigation performance of the RLPlanNav, PPO and REL-DDPG algorithms for 100 experiments in SCENE-I. (a) and (b) show the success and trapped rates with different obstacle densities, and (c) and (d) show the success and trapped rates with different map sizes.

rate. This is because in an unknown complex environment, the heuristic A* algorithm is prone to fall into a local trap.

It can be seen from Table II that the mapping and planning algorithm can obtain smooth and energy-saving trajectories in successful navigation scenarios. The proposed RLPlanNav belongs to the reactive planning structure. Compared with the traditional planning algorithm, the recursive network of the DRL part can summarize the role of historical information, and the trapped rate of RLPlanNav is 1.25%, which is far lower than the trapped rate of 5.75% of the mapping and planning algorithm. In terms of the smoothness of the trajectory, the two are similar, and both can produce a trajectory with a relatively gentle speed change.

F. Generalization Capability

In the real world, the density of obstacles and the size of the map have a significant impact on UAV navigation. Therefore, we conducted a comparative analysis of the performance of the RLPlanNav algorithm, the PPO algorithm, and the REL-DDPG algorithm in SCENE-I. The experimental results are presented in Fig. 8(a) and (b). As obstacle density rises, the success rates of all three algorithms exhibit a declining trend. Notably, the RLPlanNav algorithm maintains a success rate of approximately 90%, while the least performing PPO algorithm only achieves a 67% success rate when obstacle density reaches 60%. Furthermore, in the assessment of the trapping rates of the three algorithms, the RLPlanNav algorithm's capacity to select local target points

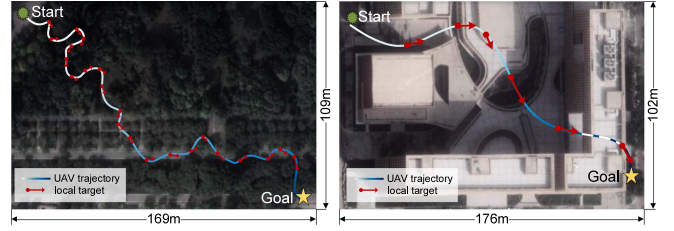


Fig. 9. Performance of the RLPlanNav algorithm in woods and buildings.

using its upper-level RL algorithm allows it to avoid getting trapped. The REL-DDPG algorithm, leveraging its experiential learning capabilities, can also remember simple environmental structures, resulting in low trapping rates for both of these methods. In contrast, the trapping rate of the PPO algorithm increases as obstacle density rises, reaching approximately 10%.

Additionally, we conducted flight tests in different environmental areas by increasing the dimensions of the environment, as depicted in Fig. 8(c) and (d). As the environmental area expands, the success rates of all three algorithms experience varying degrees of decline. Nevertheless, even in the largest environmental area, the RLPlanNav algorithm continues to maintain an impressive success rate of approximately 82%. In extensive maps where the target is distant from the starting point, the guiding role of local target points becomes even more prominent. The PPO and REL-DDPG algorithms, as end-to-end approaches that directly map flight actions from sensor information, demonstrate less effective planning abilities for long-distance targets compared to the RLPlanNav algorithm.

G. Real-World Experiments

In this section, we assessed the navigation performance and capabilities of UAVs employing the proposed algorithm in unknown real-world environments. Navigation strategies were trained in a simulated environment and then applied to real UAVs with limited perceptual range. To simplify calculations and focus on obstacle avoidance performance, the UAV's flight altitude was fixed during the experiments. The experiments were conducted in settings involving large buildings and wooded areas. The overarching goal was to establish a distant target and observe whether the UAVs could smoothly reach this global objective through the generation of local target strategies step by step.

The experimental results, depicted in Fig. 9, reveal that UAVs demonstrated efficient navigation through large buildings, avoiding entrapment in local obstacles while maintaining an impressive average flight speed of 2.1 m/s. However, the wooded environments presented distinct challenges. The presence of dynamic factors, such as tree branches swaying due to the

airflow generated by the UAVs, introduced unpredictability into the environment, disrupting the algorithms' ability to establish local targets reliably. Additionally, the perceived quality of the UAVs in these real-world environments impacted the quality and accuracy of the observations on which the reinforcement learning algorithms relied. These limitations posed challenges for the algorithms' capacity to accurately estimate the state of the environment. As a result, the overall speed of the UAVs in wooded environments was notably affected, resulting in an average speed of 0.9 m/s.

VI. CONCLUSION

This letter proposed RLPlanNav, a combined learning and traditional planner-based navigation framework for navigating in unknown environments with static obstacles. RLPlanNav contains two advantages to make it perform better, learning complex sub-goal predictions from high-dimensional observations via RL, and precise low-level control execution via a motion planner. Compared with learning-based end-to-end methods and geometric mapping-based methods, RLPlanNav can perform better and more reliable in the invisible aspects of the target. The policies generated after training with RLPlanNav are stable and can be transferred to different motion planners.

Nonetheless, our approach exhibits limitations when operating in dynamic environments featuring moving obstacles, highlighting the need to integrate object detection and accurate position estimation for optimal adaptation of the hierarchical algorithm. Furthermore, to fully harness UAV potential in complex tasks, implementing information-sharing and collaborative navigation among multi-UAV is vital. In the future, we aim to integrate multi-agent reinforcement learning with hierarchical navigation to enhance the performance of multiple UAVs in dynamic scenarios, promising improved capabilities in navigating challenging and dynamic environments.

REFERENCES

- [1] M. Mohanan and A. Salgoankar, "A survey of robotic motion planning in dynamic environments," *Robot. Auton. Syst.*, vol. 100, pp. 171–185, 2018.
- [2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [4] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver, "Memory-based control with recurrent neural networks," 2015, *arXiv:1512.04455*.
- [5] H. X. Pham, H. M. La, D. Feil-Seifer, and L. Van Nguyen, "Reinforcement learning for autonomous UAV navigation using function approximation," in *Proc. IEEE Int. Symp. Saf. Secur. Rescue Robot.*, 2018, pp. 1–6.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2022.
- [7] T. Siméon, J.-P. Laumond, and C. Nissoux, "Visibility-based probabilistic roadmaps for motion planning," *Adv. Robot.*, vol. 14, no. 6, pp. 477–493, 2000.
- [8] S. M. LaValle et al., "Rapidly-exploring random trees: Progress and prospects," *Algorithmic Comput. Robot.: New Directions*, vol. 5, pp. 293–308, 2001.
- [9] F. Gao and S. Shen, "Online quadrotor trajectory generation and autonomous navigation on point clouds," in *Proc. IEEE Int. Symp. Saf. Secur. Rescue Robot.*, 2016, pp. 139–146.
- [10] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1474–1481, Jul. 2018.
- [11] Y. Zhu et al., "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 3357–3364.
- [12] G. Tong, N. Jiang, L. Biye, Z. Xi, W. Ya, and D. Wenbo, "UAV navigation in high dynamic environments: A deep reinforcement learning approach," *Chin. J. Aeronaut.*, vol. 34, no. 2, pp. 479–489, 2021.
- [13] S. Kim, J. Park, J.-K. Yun, and J. Seo, "Motion planning by reinforcement learning for an unmanned aerial vehicle in virtual open space with static obstacles," in *Proc. IEEE 20th Int. Conf. Control Autom. Syst.*, 2020, pp. 784–787.
- [14] H. Zijian, G. Xiaoguang, W. Kaifang, Z. Yiwei, and W. Qianglong, "Relevant experience learning: A deep reinforcement learning method for UAV autonomous motion planning in complex unknown environments," *Chin. J. Aeronaut.*, vol. 34, no. 12, pp. 187–204, 2021.
- [15] K. Rana, B. Talbot, V. Dasagi, M. Milford, and N. Sünderhauf, "Residual reactive navigation: Combining classical and learned navigation strategies for deployment in unknown environments," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 11493–11499.
- [16] A. Faust et al., "PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 5113–5120.
- [17] S. Bansal, V. Tolani, S. Gupta, J. Malik, and C. Tomlin, "Combining optimal control and learning for visual navigation in novel environments," in *Proc. Conf. Robot Learn.*, 2020, pp. 420–429.
- [18] F. Xia, C. Li, R. Martín-Martín, O. Litany, A. Toshev, and S. Savarese, "ReLMoGen: Integrating motion generation in reinforcement learning for mobile manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4583–4590.
- [19] A. P. Kalidas, C. J. Joshua, A. Q. Md, S. Basheer, S. Mohan, and S. Sakri, "Deep reinforcement learning for vision-based navigation of UAVs in avoiding stationary and mobile obstacles," *Drones*, vol. 7, no. 4, 2023, Art. no. 245.
- [20] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [21] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.